

Day 1 Practice - 15 Questions

Topics: array creation | dtype / astype | attributes | indexing | slicing | boolean indexing | Level: Medium | Rule: no loops!

- Q1.** Create an array of the first 12 **odd numbers** using a single **np.arange** call.
- Q2.** Using **np.linspace**, create 9 equally spaced values from 5 to 45. Before running, **predict the gap** between consecutive values - then verify.
- Q3.** Create **np.arange(1, 10)** and **np.linspace(1, 9, 9)**. Print both and compare their **dtype**. Why are they different?
- Q4.** Create a **6x6 identity matrix** with integer dtype - the output must show no decimal points.
- Q5.** Create a **3x4 matrix** where every element is **5**, and report its shape, size, and ndim.
- Q6.** Given **a = np.array([2, 4, 6])** and **b = np.array([2.0, 4.0, 6.0])** - check both dtypes, then convert b to int with **astype** and verify the dtypes now match.
- Q7.** **Predict the exact output** of **np.array([10.9, 25.5, 39.99]).astype(int)** before running it. What rule does the result prove?
- Q8.** Given **arr = np.arange(10, 101, 10)** - extract: (a) the 4th element, (b) the last element using negative indexing, (c) the 2nd-last element.
- Q9.** From the same arr, slice out exactly **[30 40 50 60]**, and separately produce the **full array in reverse**.
- Q10.** Given **arr = np.arange(1, 21)** - extract **every 3rd element starting from index 1** (result: 2, 5, 8, ...).
- Q11.** Using **one slice only**, extract everything from arr (1-20) EXCEPT the first two and the last two elements.
- Q12.** Using the matrix below, extract: (a) the **3rd row**, (b) the **last column**, (c) the **centre 3x3 block**.

```
m = np.array([[ 1,  2,  3,  4,  5],
              [ 6,  7,  8,  9, 10],
              [11, 12, 13, 14, 15],
              [16, 17, 18, 19, 20],
              [21, 22, 23, 24, 25]])
```

- Q13.** From the same matrix, extract the **four corner elements** as a 2x2 array - using **step slicing only**.
- Q14.** From the same matrix, extract **every alternate row AND every alternate column** at the same time (the result is 3x3).
- Q15.** Given the marks array below - (a) extract all marks **greater than 60**; (b) extract marks **between 50 and 80** using two conditions with & (remember the parentheses!).

```
marks = np.array([56, 91, 44, 77, 62, 85, 39, 70])
```

NOTE: Cross-check every slicing answer by predicting the output BEFORE running the cell - that habit is what builds real logic.